

Threads und Nebenläufigkeit



Quelle

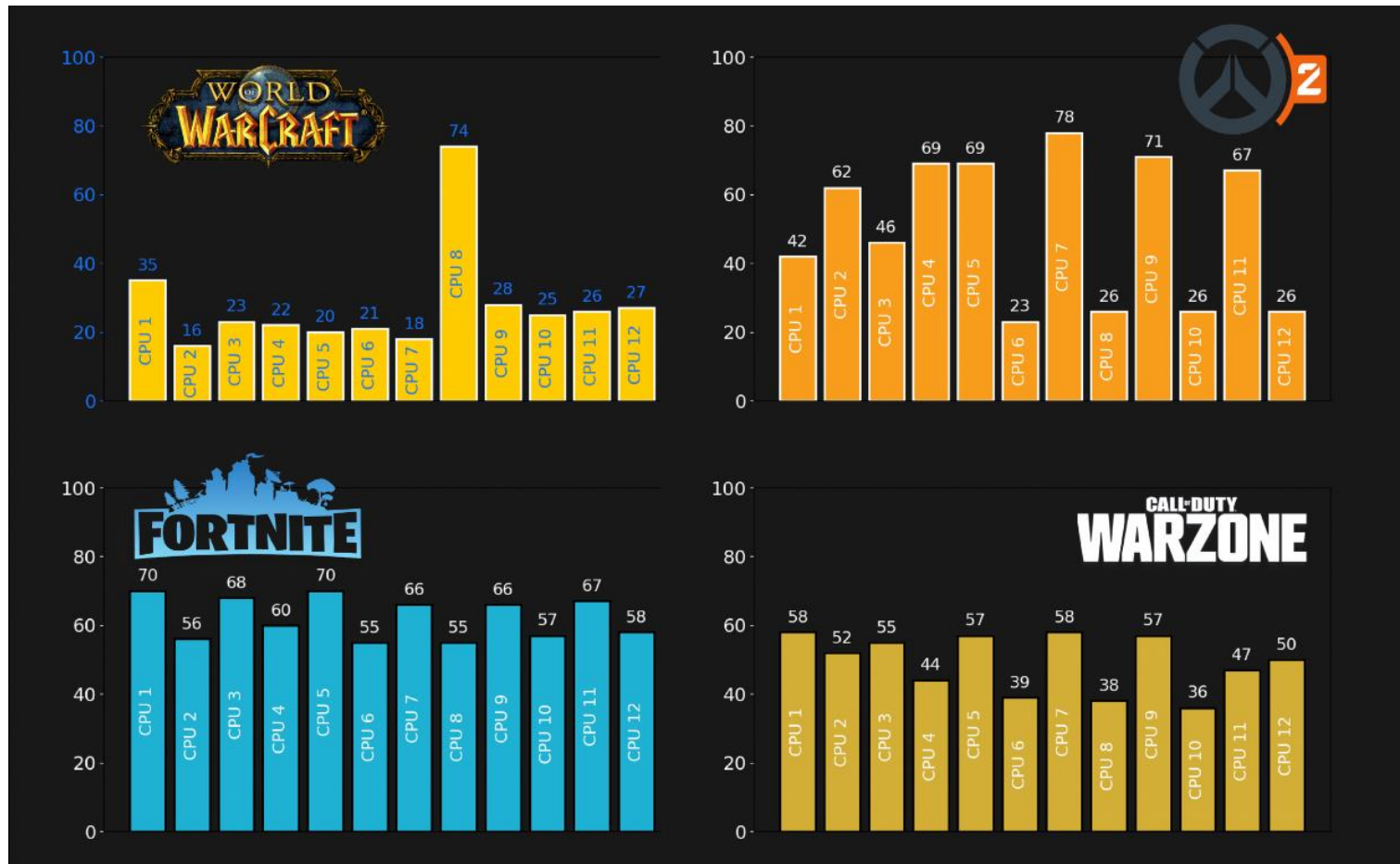
Nebenläufigkeit



Quelle

Optimiert für Multi-Threaded?

Die meisten Anwendungen nutzen inzwischen mehrere CPU-Kerne, sind aber nicht immer für Multi-Threaded optimiert. Die nachfolgende Grafik zeigt zum Beispiel die Auslastung der einzelnen CPU-Kerne für verschiedene Videospiele.



Nebenläufigkeit: Vorteile

- Schnellere Berechnungen 🕒
z. B. mehrere Rechenkerne teilen sich die Berechnung, um sie zu beschleunigen.
- Programme reagieren schneller 🔄
z. B. GUI blockiert nicht, bis eine Berechnung fertig ist.
- Im Hintergrund Warten auf Ereignisse ⚡
z. B. Callbacks und Futures 🧐
- Faire Ressourcenverteilung ⚖️
z. B. mehrere User greifen gleichzeitig auf eine Webseite / Internetstream zu.

Thread: Definition



Ein Thread ("Faden") ist ein Ausführungsstrang in einem Programm.

Die virtuelle Java-Maschine ermöglicht es einer Anwendung, **mehrere Ausführungsstränge gleichzeitig** (nebenläufig bzw. parallel) laufen zu lassen.

D. h. eine App kann auf **mehrere Rechenkernen** (z. B. CPUs) **nebenläufig** laufen.



Multi-Threading vs Nebenläufigkeit

Nebenläufigkeit (Concurrent)

Nebenläufigkeit beschreibt die Fähigkeit eines Systems, **mehrere Aufgaben gleichzeitig** zu bearbeiten.

Aufgaben können auf einem einzelnen CPU-Prozessor durch **Wechseln zwischen Aufgaben** (Task-Switching) umgesetzt werden.

Multi-Threading

Multi-Threading ist eine **Technik zur Implementierung von Nebenläufigkeit**.

Hierbei wird ein **Prozess** (Anwendung) **in mehrere Threads aufgeteilt**, die unabhängig oder miteinander interagierend ausgeführt werden.

Multithreading nutzt die **Parallelität eines Mehrkernprozessors**, um Threads tatsächlich **gleichzeitig** auf verschiedenen Kernen auszuführen (falls verfügbar).

My Life with Unexpected Threads

Wenn eine Java-App startet, wird ein non-daemon Thread erzeugt, der die Methode **main** aufruft.

Mit anderen Worten: **du hast die ganze Zeit Threads verwendet!** 🤪

Die JVM fährt mit der Ausführung von Threads fort, bis einer der folgenden Fälle eintritt:

1. Die exit-Methode der Klasse Runtime wurde aufgerufen.
2. Alle non-daemon Threads sind gestorben: †
 - entweder durch Beenden der run-Methode (gleich mehr dazu...)
 - oder durch das Auslösen einer nicht abgefangenen Exception. 🐛

Der MainThread

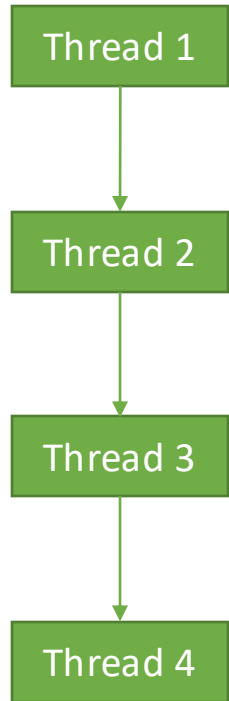
```
class SayHelloMainThread {  
  
    public static void main(String[] args) {  
        System.out.println("Hello MainThread!");  
    }  
}
```



Ick bins, der
MainThread!

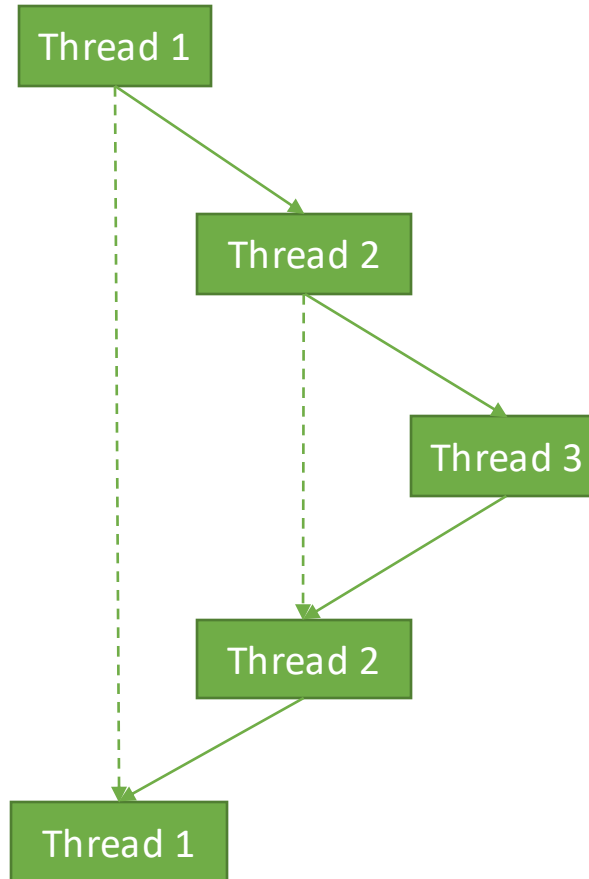
Nebenläufigkeit: Sequenziell vs Parallel 1/2

Sequenziell



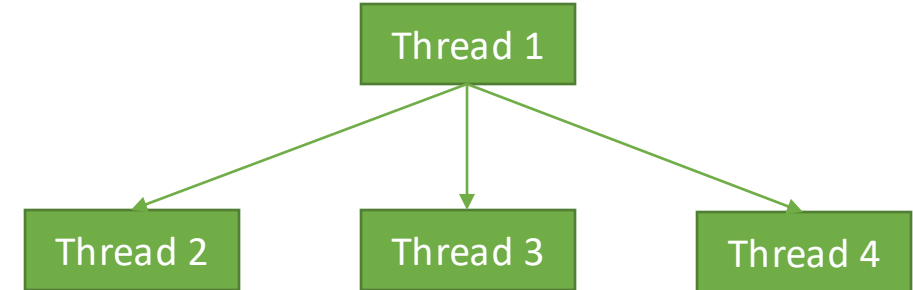
Jeder Thread läuft unterbrechungsfrei **nacheinander** ab.

Verzahnt



Unterbrechen und häufiger Wechsel der Threads erzeugt **scheinbare Parallelität**.

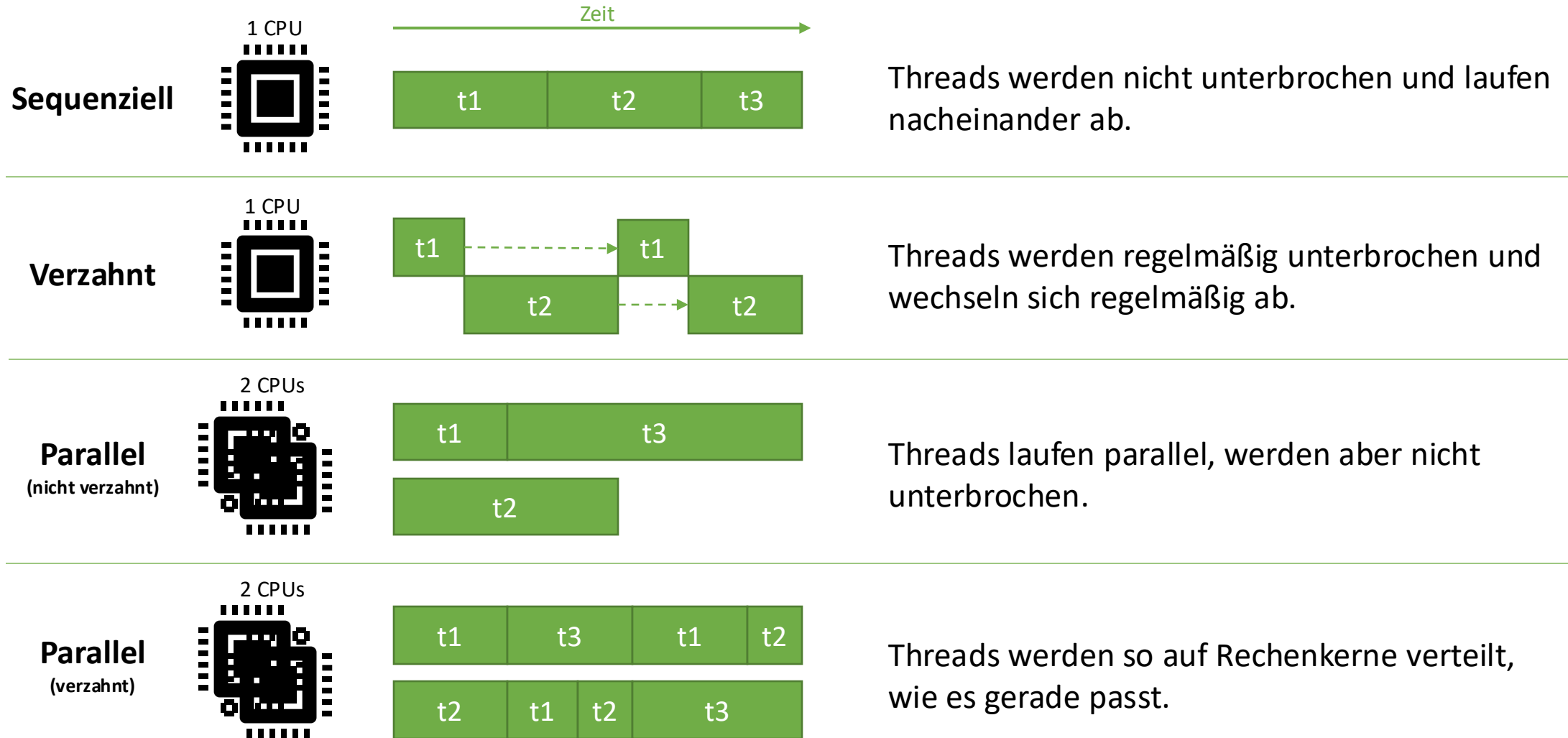
Parallel (nebenläufig)



Threads laufen **gleichzeitig parallel**. Nur möglich, wenn **mehrere Rechenkerne** vorhanden sind.

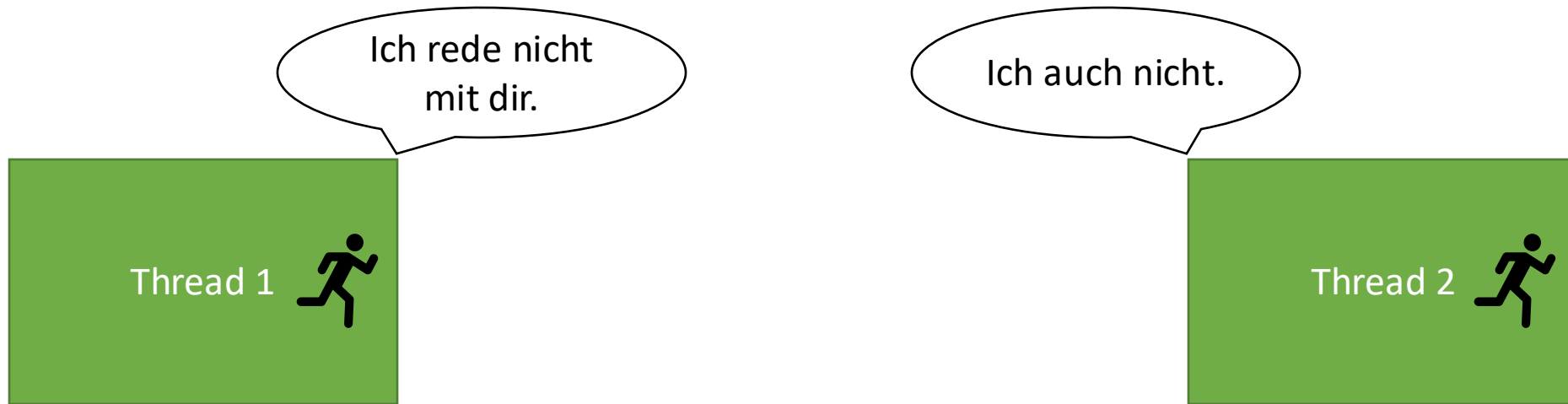
Hinweis: Multi-Threading ist auch auf CPUs mit nur einem Kern möglich. Die Threads werden dann verzahnt ausgeführt.

Nebenläufigkeit: Sequenziell vs Parallel 2/2



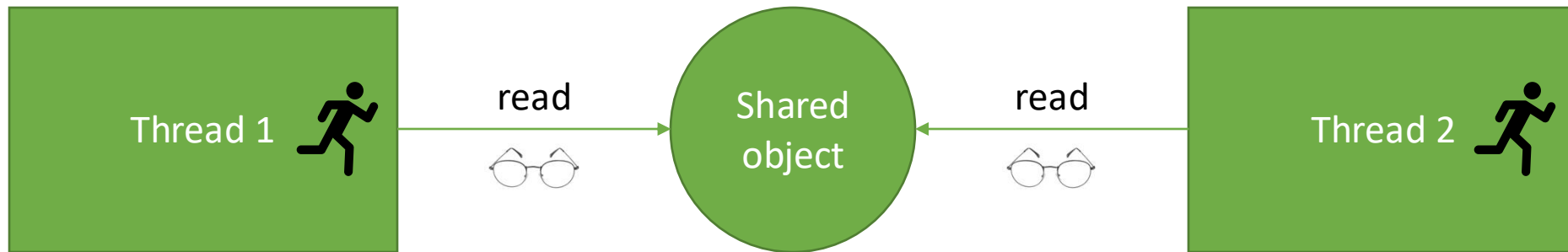
Nebenläufigkeit: Best Case #1

Zwei (oder mehr) Threads laufen **unabhängig** voneinander.



Nebenläufigkeit: Best Case #2

Zwei (oder mehr) Threads greifen **nur lesend** auf gemeinsame Ressourcen zu.



Je mehr Prozesse und Daten immutable sind, desto größer das Potenzial für Nebenläufigkeit!

#KeineSeiteneffekte, #PureLambda, #ImmutableCollections, #Kapselung, #WenigSetter, #record

```

public class UserContext {
    // DER FEIND: Eine veränderbare, statische Variable
    public static String currentUser;

    public static void processRequest(String user) {
        // 1. Speichern, wer gerade anfragt
        currentUser = user;

        // ... hier passiert Logik, Datenbankzugriffe etc. ...
        // Wir simulieren das mit Sleep (Verzögerung)
        try { Thread.sleep(100); } catch (Exception e) {}

        // 2. Bestätigung senden
        System.out.println("Daten gesendet an: " + currentUser);
    }
}

```

Warum ist das so schlimm?

Unvorhersehbarkeit: Der Fehler tritt nur bei hoher Last auf (Race Condition). Beim lokalen Testen (nur du allein) funktioniert alles perfekt.

Globale Zerstörung: Da die Variable static ist, kann *jeder* Teil deines Codes diesen Zustand kaputt machen.

Wie macht man es richtig?

Vermeide veränderbare static-Variablen für Zustände, die zur Laufzeit variieren (wie User-Daten, Request-Daten, Zähler).

Lokale Variablen: Gib Daten als Parameter weiter (processRequest(String user)). Lokale Variablen liegen auf dem Stack und sind für jeden Thread isoliert.

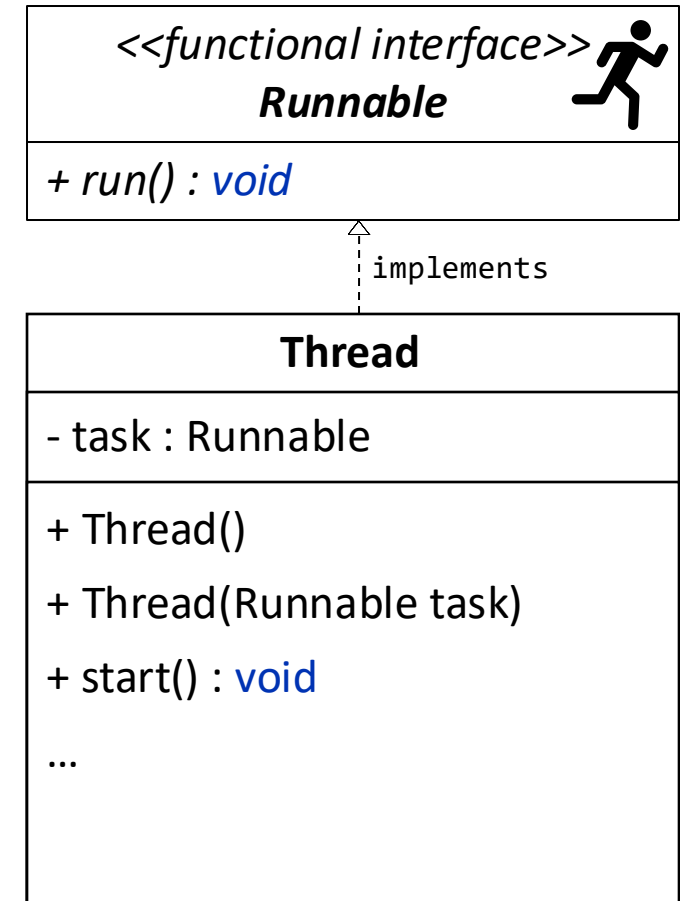
Instanz-Variablen: Erstelle für jeden Request ein neues Objekt (new RequestHandler(user)). Jeder Thread hat sein eigenes Objekt.

Thread: Option 1/3



Option 1: Erben von Thread

```
class GreetThread extends Thread {  
  
    private String greeting;  
  
    GreetThread(String greeting) { this.greeting = greeting; }  
  
    @Override  
    public void run() {  
        System.out.println(greeting);  
    }  
  
    public static void main(String[] args) {  
        GreetThread greetThread = new GreetThread("Hi!"); // create a new Thread  
        greetThread.start(); // executes run() as a Thread  
    }  
}
```



Siehe: <https://docs.oracle.com/en/java/javase/23/docs/api/java.base/java/lang/Thread.html>

Thread: Option 2/3



Option 1: Erben von Thread

```
class GreetThread extends Thread {  
  
    private String greeting;  
  
    GreetThread(String greeting) { this.greeting = greeting; }  
  
    @Override  
    public void run() {  
        System.out.println(greeting);  
    }  
  
    public static void main(String[] args) {  
        GreetThread greetThread = new GreetThread("Hi!");  
        greetThread.start();  
    }  
}
```

Option 2: Implementieren von Runnable



```
class GreetRun implements Runnable {  
  
    private String greeting;  
  
    GreetRun(String greeting) { this.greeting = greeting; }  
  
    @Override  
    public void run() {  
        System.out.println(greeting);  
    }  
  
    public static void main(String[] args) {  
        GreetRun greetRun = new GreetRun("Hi!");  
        new Thread(greetRun).start();  
    }  
}
```

*executes run()
as a Thread*



Vorteil: Die Klasse GreetRun kann von einer anderen Klasse erben.



FYI: Da Thread-Methoden hier nicht vererbt werden, sind sie über Thread.currentThread() aufrufbar.

Merkmal	Option 1 (extends Thread)	Option 2 (implements Runnable)
Philosophie	Die Klasse ist ein Arbeiter.	Die Klasse definiert Arbeit für einen Arbeiter.
Vererbung	Sackgasse: Da du schon von Thread erbst, kannst du von keiner anderen Basisklasse mehr erben.	Flexibel: Du kannst implements Runnable nutzen und gleichzeitig extends MyBaseClass machen.
Kapselung	Vermischt: Die Geschäftslogik (run) und die Thread-Verwaltung (start, interrupt) sind im selben Objekt.	Sauber getrennt: Der Thread kümmert sich ums Starten/Stoppen, dein Runnable nur um die Logik.
Zugriff auf Thread-Methoden	Einfach: Du kannst this.getName() oder this.sleep() aufrufen, da du ja im Thread-Objekt bist.	Etwas umständlicher: this ist nur deine Aufgabe. Um an den Thread zu kommen, musst du Thread.currentThread() aufrufen.
Wiederverwendbarkeit	Schlecht: Der Thread ist an diese eine Aufgabe gebunden.	Gut: Dasselbe Runnable-Objekt könnte theoretisch von verschiedenen Threads ausgeführt werden (z.B. in einem ThreadPool).

Thread: Option 3/3



Option 3: Lambda (quasi wie Option 2)

```
public static void main(String[] args) {  
    Thread t1 = new Thread(() -> System.out.println("Hi!"));  
    t1.start(); // executes run() as a Thread  
  
    Thread t2 = Thread.ofPlatform().factory().newThread(() -> System.out.println("Hi!"));  
    t2.start();  
  
    Thread.ofPlatform().start(() -> System.out.println("Hi!")); // directly starts a new Thread  
}
```

```
@FunctionalInterface  
public interface Runnable {  
  
    void run();  
}
```

Virtuelle Threads

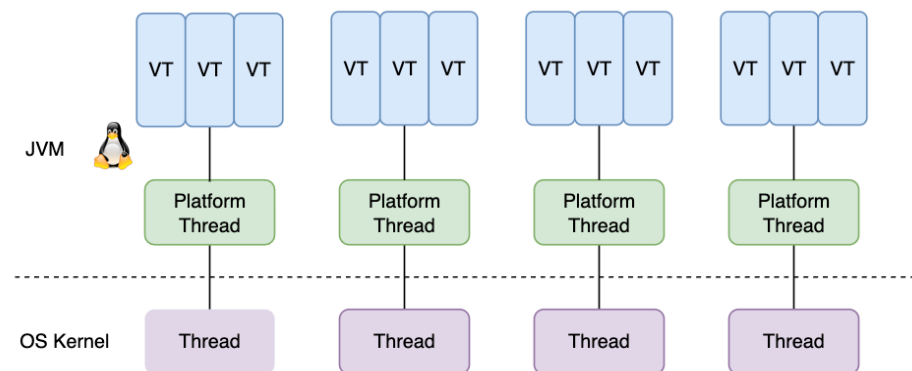


Threads werden vom Betriebssystem verwaltet. Solche **Plattform-Threads** sind schwergewichtig, da sie viel Speicher benötigen und ihre Erstellung sowie der Kontextwechsel teuer sind. Die Anzahl gleichzeitig laufender Plattform-Threads ist daher begrenzt – in der Praxis oft auf etwa **5.000 bis 10.000**.

Mit **JDK 21** wurden **virtuelle Threads** eingeführt, um die gleichzeitige Ausführung von sehr vielen Aufgaben zu ermöglichen. Bei virtuellen Threads erfolgt die Verwaltung durch die JVM, was interne Optimierungen bei Scheduling und Ressourcenmanagement ermöglicht. Virtuelle Threads sind daher:

- **Leichtgewichtig:** geringerer Speicherverbrauch, schneller Kontextwechsel, einfache Erstellung und Zerstörung.
- **Skalierbar:** Millionen von virtuellen Threads können gleichzeitig laufen.
- **Verwenden dieselbe API:** Die Programmier-API bleibt dieselbe wie für Plattform-Threads (siehe Thread und Runnable).
- **Blockieren effizient:** Sie können blockierende Operationen (z. B. I/O) effizient ausführen, da sie bei Blockierung automatisch vom Scheduler geparkt und anderen Aufgaben Platz machen.

```
public static void main(String[] args) {  
    Thread v1 = Thread.ofVirtual().factory().newThread() -> System.out.println("Hi!");  
    v1.start(); // executes run() as a virtual Thread  
  
    Thread.ofVirtual().start() -> System.out.println("Hi!"); // directly starts a virtual Thread  
}
```



Thread: Methoden

Eine kleine Auswahl an Methoden aus der Klasse [Thread](#):

// Definiert, was der Thread tun soll. 
`public void run()` *//Nicht direkt aufrufen!*

// Starte den Thread und führe seine run()-Methode nebenläufig aus. 
`public void start()`

// Der aktuelle Thread wartet bis der Thread, auf dem join() gerufen wurde, stirbt. 
`public void join()` *throws* `InterruptedException`

// Der aktuelle Thread legt sich für die definierte Dauer in Millisekunden schlafen. 
`public static void sleep(long millis)` *throws* `InterruptedException`

*// Versucht einen Thread zu unterbrechen (sehr unfreundlich), indem im Thread eine **InterruptedException** geworfen wird. Der Thread kann auf die Exception reagieren, sie aber auch ignorieren (auch unfreundlich).*
`public void interrupt()` 

Thread: States

New

Thread has not yet been started.

Runnable

Thread was started and is waiting to be executed.

Running

Thread is executed in the *Java Virtual Machine* (JVM).

Blocked

Thread is waiting for a monitor / mutex lock.

Waiting

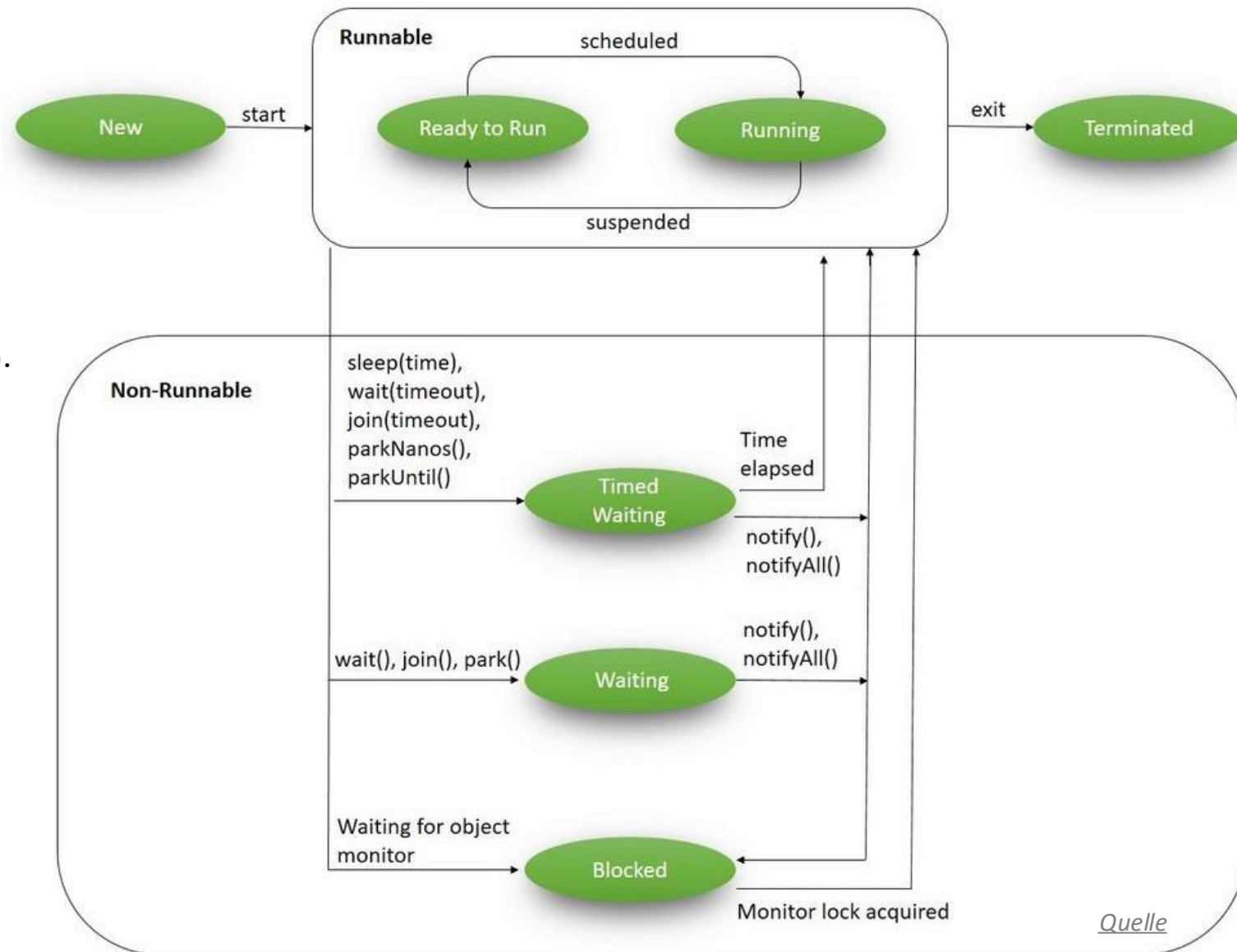
Thread is waiting indefinitely for another thread to perform a particular action.

TimedWaiting

Thread is waiting for another thread to perform an action for up to a specified waiting time.

Terminated

Thread has ended and was terminated.



Quelle

Nicht-Determinismus 1/2

```
class TicToc {  
    public static void main(String[] args) throws Exception {  
  
        Thread tic = new Thread(() -> {  
            for (int i = 0; i < 20; i++) {  
                System.out.print("tic ");  
            }  
        });  
  
        Thread toc = new Thread(() -> {  
            for (int i = 0; i < 20; i++) {  
                System.out.print("toc ");  
            }  
        });  
  
        tic.start();  
        toc.start();  
  
        tic.join();  
        toc.join();  
  
        System.out.println("fertig!");  
    }  
}
```

Was wird ausgegeben?



Nicht-Determinismus 2/2

```
class TicToc {  
    public static void main(String[] args) throws Exception {  
  
        Thread tic = new Thread(() -> {  
            for (int i = 0; i <= 20; i++) {  
                System.out.print("tic ");  
            }  
        });  
  
        Thread toc = new Thread(() -> {  
            for (int i = 0; i <= 20; i++) {  
                System.out.print("toc ");  
            }  
        });  
  
        tic.start();  
        toc.start();  
  
        tic.join();  
        toc.join();  
  
        System.out.println("fertig!");  
    }  
}
```

Es gibt **keine Garantie in welcher Abfolge Threads ausgeführt werden** (nicht-deterministisch bzw. nicht vorhersehbar).

FYI: Mit Thread.sleep, Thread.wait, Thread.join, Thread.priority, Thread.yield lässt sich das Scheduling beeinflussen.

Mögliche Ausgabe:

```
toc toc toc toc toc toc tic tic tic tic tic tic tic  
tic toc toc toc toc toc toc toc toc tic tic tic tic  
tic tic tic tic tic tic tic tic tic toc toc toc toc  
toc toc toc fertig!
```

Warum Nicht-Determinismus?



Quelle

Im Gegensatz zu sequenziellen (serialisierten) Programmen ist die Ausführungsreihenfolge bei nebenläufigen Programmen nicht vorhersagbar (nicht-deterministisch). Warum?

Die **Ausführungsreihenfolge** mehrerer Threads – und damit das Ergebnis – hängt von **unterschiedlichen Faktoren** ab:

- Das Auftreten von **unvorhersehbaren Events** (z. B. Mausklick, Nachrichteneingang, Exceptions).
- Die **Anzahl** der **Threads** und der **verfügbaren CPU-Kerne**.
- Die **Geschwindigkeit** und **Auslastung** der verfügbaren **Kerne/Hardware** (z. B. I/O, Bandbreite).
- Das **Schedulingverfahren** von Threads (z. B. **Priorisierung**, **Time-Slices**):
 - Damit ein Thread nicht ewig läuft und alle anderen blockiert, bekommt jeder Thread einen sogenannten Time-Slice, der bestimmt, wie lange ein Thread am Stück laufen darf.
 - Die Dauer des Time-Slice und die CPU-Geschwindigkeit bestimmt, wie viel Operationen bis zum nächsten Thread-Wechsel ausgeführt werden können.

Threads sind ein **Software-Konzept**, das auf jeder Computer-Plattform funktionieren soll unabhängig von der Anzahl der vorhandenen Kerne/Hardware. Die Ausführung der Threads kann sich aber je nach Plattform oder Betriebssystem unterscheiden.

Warum nicht immer Nebenläufigkeit?

Siehe Titel ↑



Nebenläufigkeit: Nachteile



1. Koordination

z. B. gemeinsamer Zugriff muss mit Sorgfalt gestaltet werden.

2. Verwaltungsaufwand

z. B. ständiger Kontextwechsel: Wiederherstellen des alten Thread-Zustands.

3. Debugging

z. B. Breakpoints nur für einen Thread möglich, alle anderen Threads laufen weiter.

4. Unit Tests

z. B. Ergebnisse in zufälliger Reihenfolge oder Exceptions (aus anderen Threads) können nicht abgefangen werden.

5. Race Conditions

z. B. Ergebnis hängt von der Ausführungsreihenfolge ab.

6. Lost Update

z. B. Änderung wird sofort überschrieben und geht dadurch verloren.

7. Deadlocks

z. B. zirkuläre Abhängigkeit auf geteilte Ressourcen.

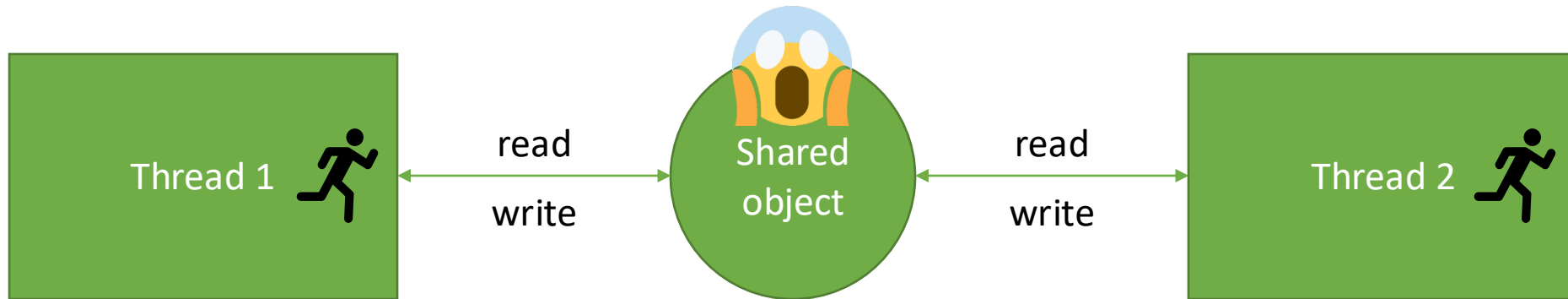


8. Bugs!

Bugs treten sporadisch auf und lassen sich schwer reproduzieren.

Nebenläufigkeit: Das Grundproblem

Zwei (oder mehr) Threads greifen **schreibend und lesend** auf **dasselbe** Objekt bzw. **dieselbe** Ressource zu.



Quelle

Race Condition

Race Condition ist ein Zustand eines Programms, dessen **Verhalten** vom **relativen Timing** oder der **Verzahnung** mehrerer Threads **abhängt**.

Je nach **Ausführungsreihenfolge**, können **mehrere mögliche Ergebnisse** auftreten.

Thread-safe beschreibt ein Programm, einen Code oder eine Datenstruktur, die frei von Race Conditions ist, wenn mehrere Threads gleichzeitig darauf zugreifen.

Race Condition++

Ein harmloses Beispiel...

```
void increment() {  
    value++;  
}
```

value++ ist **keine atomare Operation** sondern besteht aus zusammengesetzten Anweisungen:

```
value = value + 1;
```

Die Operation besteht aus folgenden Teilschritten:

1. aktuellen Wert aus value lesen.
2. value um 1 erhöhen.
3. neuen Wert in value schreiben.

Nach **jedem** Teilschritt kann der Thread von einem anderen Thread unterbrochen werden!

Race Condition 1/2

```
Counter sharedCounter = new Counter(0); // value = 0;
var t1 = new Thread(() -> sharedCounter.increment()); // value++;
var t2 = new Thread(() -> sharedCounter.increment()); // value++;
t1.start();
t2.start();
```

Möglicher Ablauf 1

t1	t2	value (in sharedCounter)
Read		0
Write		1 <- (0+1)
	Read	1
	Write	2 <- (1+1)

```
t1.join(); // wait for t1 to finish.
t2.join(); // wait for t2 to finish.
sharedCounter.value(); // returns 2
```



```
class Counter {
    private int value;

    Counter(int value) {
        this.value = value;
    }

    int value() {
        return value;
    }

    void increment() {
        value++;
    }
}
```

Race Condition 2/2

```
Counter sharedCounter = new Counter(0); // value = 0;
var t1 = new Thread(() -> sharedCounter.increment()); // value++;
var t2 = new Thread(() -> sharedCounter.increment()); // value++;
t1.start();
t2.start();
```

Möglicher Ablauf 2

t1	t2	value (in sharedCounter)
Read		0
	Read	0
Write		1 <- (0+1)
	Write	1 <- (0+1)

```
t1.join(); // wait for t1 to finish.
t2.join(); // wait for t2 to finish.
sharedCounter.value(); // returns 1 but should be 2! #WTF
```



```
class Counter {
    private int value;

    Counter(int value) {
        this.value = value;
    }

    int value() {
        return value;
    }

    void increment() {
        value++;
    }
}
```

Synchronized

```
class SayNoToRaceCondition {  
    private static final int TO_HUNDRED = 100;  
    private static final Counter sharedCounter = new Counter(0);  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_HUNDRED));  
        var t2 = new Thread(count(TO_HUNDRED));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                synchronized (sharedCounter) { // synchronized schützt sharedCounter vor Race Conditions!  
                    sharedCounter.increment(); // value++;  
                }  
                System.out.println(sharedCounter.value());  
            }  
        };  
    }  
}
```



```
class Counter {  
  
    private int value;  
  
    Counter(int value) {  
        this.value = value;  
    }  
  
    int value() {  
        return value;  
    }  
  
    void increment() {  
        value++;  
    }  
}
```


Synchronized

```
class SayNoToRaceCondition {  
    private static final int TO_HUNDRED = 100;  
    private static final Counter sharedCounter = new Counter(0);  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_HUNDRED));  
        var t2 = new Thread(count(TO_HUNDRED));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                synchronized (sharedCounter) { // synchronized schützt sharedCounter vor Race Conditions!  
                    sharedCounter.increment(); // value++;  
                }  
                System.out.println(sharedCounter.value());  
            }  
        };  
    }  
}
```



```
class Counter {  
  
    private int value;  
  
    Counter(int value) {  
        this.value = value;  
    }  
  
    int value() {  
        return value;  
    }  
  
    void increment() {  
        value++;  
    }  
}
```

Synchronized: Critical Section

```
class SayNoToRaceCondition {  
    private static final int TO_HUNDRED = 100;  
    private static final Counter sharedCounter = new Counter(0);  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_HUNDRED));  
        var t2 = new Thread(count(TO_HUNDRED));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                 synchronized (sharedCounter) {  
                    sharedCounter.increment(); // value++;  
                }  
                System.out.println(sharedCounter.value());  
            }  
        };  
    }  
}
```

Mit dem Schlüsselwort **synchronized** wird ein kritischer Bereich geschützt, sodass andere Threads diesen Bereich nicht unterbrechen können ("atomarer Bereich").



Mutex: Safer Thread 1/3

Ein **Mutex** („mutual **ex**clusion“) **schützt** einen bestimmten Bereich (nicht mehr unterbrechbar).

In Java wird dies durch das Schlüsselwort **synchronized** realisiert.

```
synchronized (sharedObject) { critical section }
```

Mutex: Safer Thread 2/3

Ein **Mutex** („**mutual exclusion**“) **schützt** einen bestimmten Bereich (nicht mehr unterbrechbar).

In Java wird dies durch das Schlüsselwort **synchronized** realisiert.

- Ein Mutex sollte das **gemeinsame Objekt** sein, auf das **alle Threads zugreifen**.
- Ein Mutex sollte **final** sein.
- Verwende **keine @ValueObject** wie **Strings, Zahlen oder Primitive** als Mutex (unterschiedliche Referenz).

```
synchronized (mutex) { critical section }
```

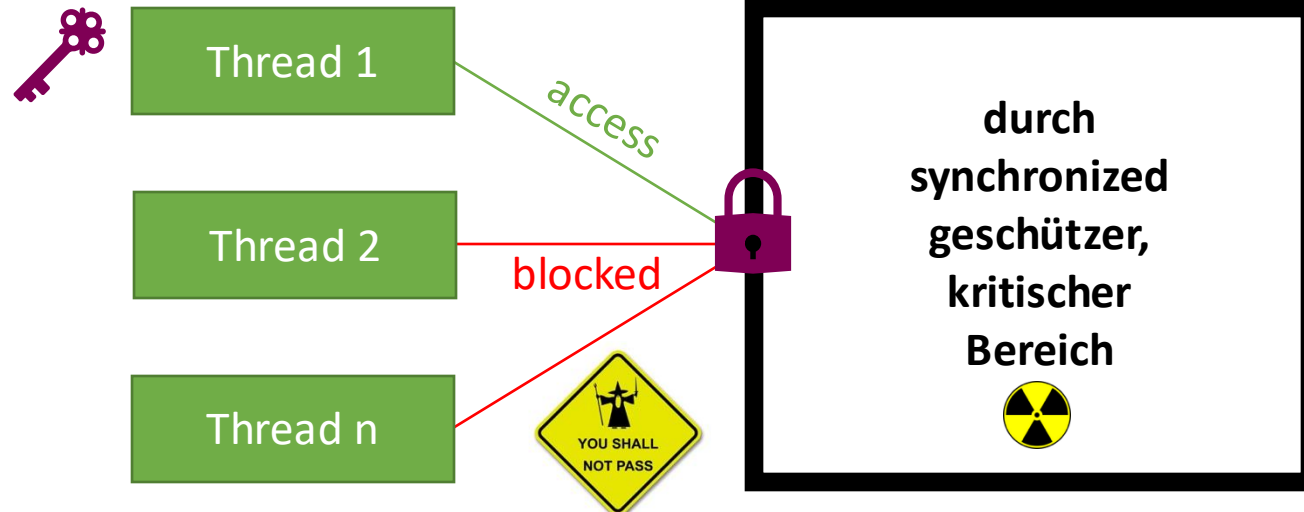
Mutex: Safer Thread 3/3

Ein **Mutex** („**mutual exclusion**“) **schützt** einen bestimmten Bereich (nicht mehr unterbrechbar). In Java wird dies durch das Schlüsselwort **synchronized** realisiert.

- Ein Mutex sollte das **gemeinsame Objekt** sein, auf das **alle Threads zugreifen**.
- Ein Mutex sollte **final** sein.
- Verwende **keine @ValueObject** wie **Strings, Zahlen oder Primitive** als Mutex (unterschiedliche Referenz).

synchronized (*mutex*) { critical section }

„Es gibt nur einen Schlüssel zum Schloss. Und nur wer den Schlüssel hat, darf rein.“



Der „Schlüssel“ wird automatisch an einem Thread vergeben, sobald er einen freien synchronized-Block betritt. Er gibt den Schlüssel automatisch wieder ab, sobald er den synchronized-Block verlässt.



<https://fabianschwarze.de/strombluete/>

Exkurs: C++/Arduino

```
// Auszug aus einem Projekt STROMBLÜTE  
// Wenn der Befehl "open" über das Netzwerk kommt:
```

```
if (eventName == "open") {
```

```
// 1. Versuche den "Schlüssel" zu bekommen (wie synchronized)  
// portMAX_DELAY bedeutet: Ich warte ewig, bis ich dran bin.
```

```
if (xSemaphoreTake(taskMutex, portMAX_DELAY) == pdTRUE) {
```

```
// --- KRITISCHER BEREICH (Critical Section) ---  
// Wenn sich die Blume schon bewegt (alter Task läuft noch),  
// muss dieser erst gestoppt werden.
```

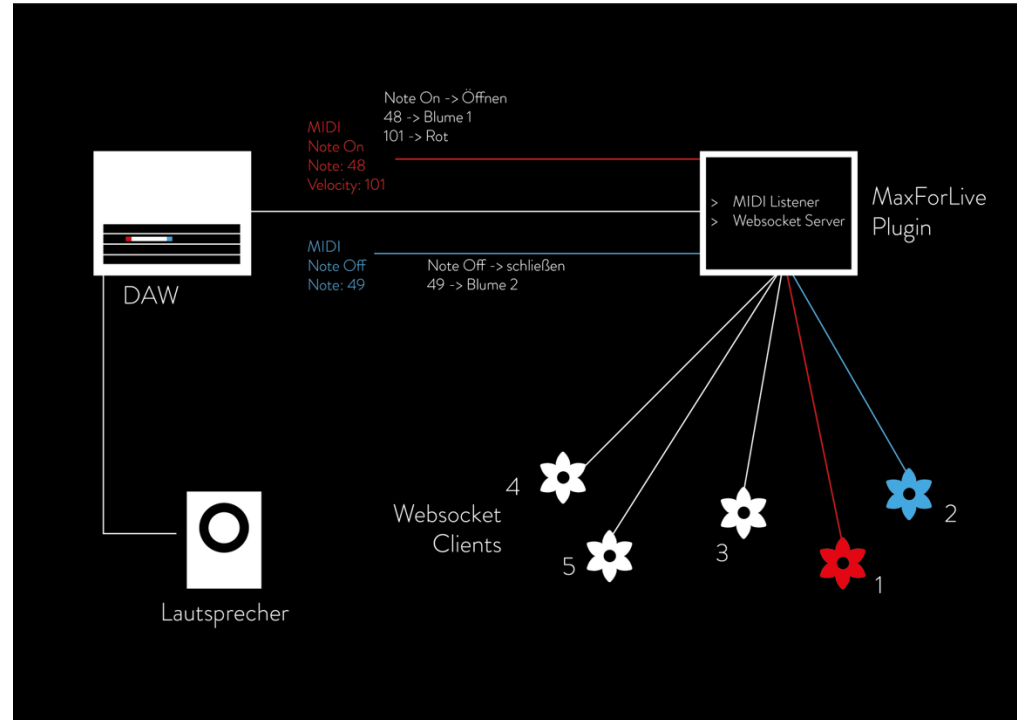
```
if (taskHandle != NULL) {  
    vTaskDelete(taskHandle);  
    taskHandle = NULL;
```

```
}
```

```
// Starte den neuen Task "openFlower"  
xTaskCreate(openFlower, "open flower", ..., &taskHandle);  
// -----
```

```
// 2. Schlüssel wieder zurückgeben  
xSemaphoreGive(taskMutex);
```

```
}  
}
```



Gemeinsamer kritischer Bereich: this

```
class SynchronizedCounter {  
    private int value = 0;
```

```
    void increment() {  
        synchronized (this) {  
            this.value++;  
        }  
    }
```

```
    void decrement() {  
        synchronized (this) {  
            this.value--;  
        }  
    }  
}
```

Ein Schloss und Schlüssel pro SynchronizedCounter-Objekt.

Common Critical Section per SynchronizedCounter-Object



access

Thread 1



blocked

Thread 2



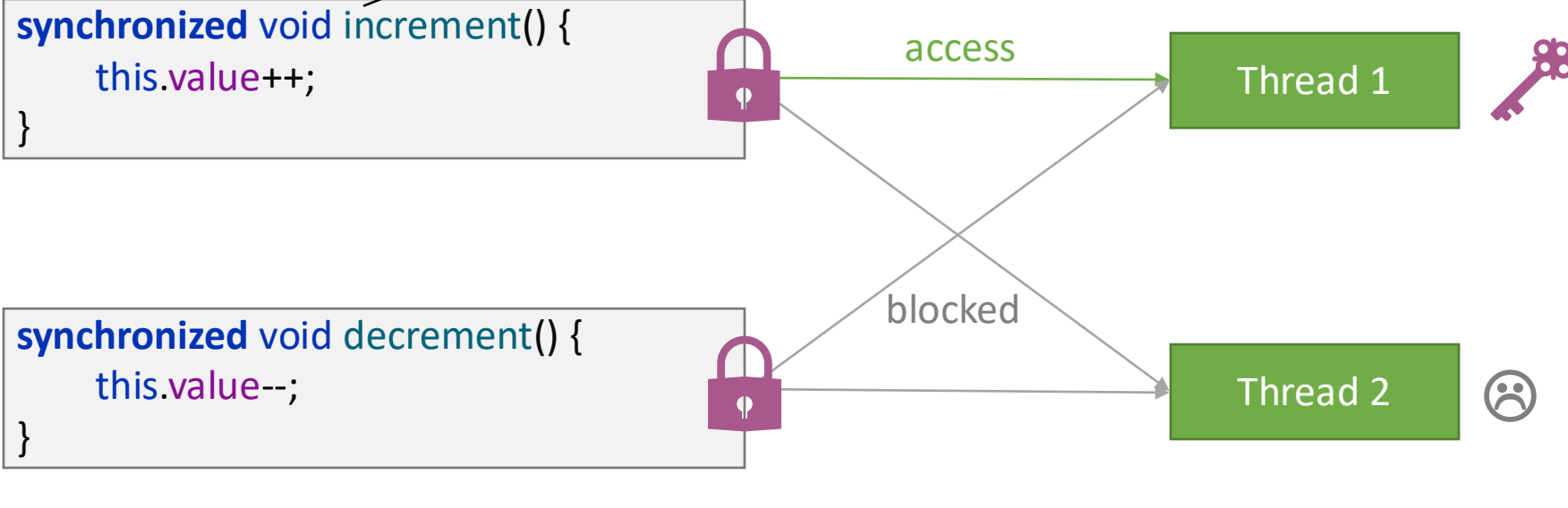
Die Bereiche in **increment()** und **decrement()** sind durch **dasselbe Objekt `this`** geschützt. In diesem Beispiel gilt die Mutex immer für **das jeweilige Objekt**. Wenn mehrere **new** `SynchronizedCounter()`-Objekte erzeugt werden, dann gibt es für **jedes dieser Objekte ein eigenes Schloss und einen eigenen Schlüssel!** Betritt ein Thread die kritische Zone von einem **Objekt**, sind **dessen** kritische Zonen **für alle anderen Threads blockiert**. Der Thread gibt den Schlüssel frei, sobald er den kritischen Bereich wieder verlässt.

Gemeinsamer kritischer Bereich: this

```
class SynchronizedCounter {  
    private int value = 0;  
      
    synchronized void increment() {  
        this.value++;  
    }  
      
    synchronized void decrement() {  
        this.value--;  
    }  
}
```

Ein Schloss und Schlüssel **pro**
SynchronizedCounter-Objekt.

Common Critical Section per
SynchronizedCounter-Object



Das Schlüsselwort `synchronized` kann auch direkt vor der Methode geschrieben werden. Dieses Beispiel hier ist eine gleichwertige alternative Schreibweise zu `synchronized(this)` (siehe vorherigen Folie).

Gemeinsamer kritischer Bereich: Object

```
class SynchronizedCounter {  
    private final Object mutex = new Object();  
    private int value = 0;
```

```
    void increment() {
```

```
        synchronized (mutex) {  
            this.value++;  
        }
```

```
    void decrement() {
```

```
        synchronized (mutex) {  
            this.value--;  
        }
```

Ein Schloss und Schlüssel pro
SynchronizedCounter-Objekt.

access

Thread 1

blocked

Thread 2

Common Critical Section per
SynchronizedCounter-Object



Als Mutex kann jedes beliebige Objekt verwendet werden. Es muss aber immer **dieselbe Referenz** haben. Für **Strings und Zahlen** ist dies **nicht garantiert**, daher sollten sie **nicht als Mutex verwenden** werden! Weiterhin ist es gute Praxis, die **Mutex-Variable final** zu machen, damit keine neue Zuweisung möglich ist.

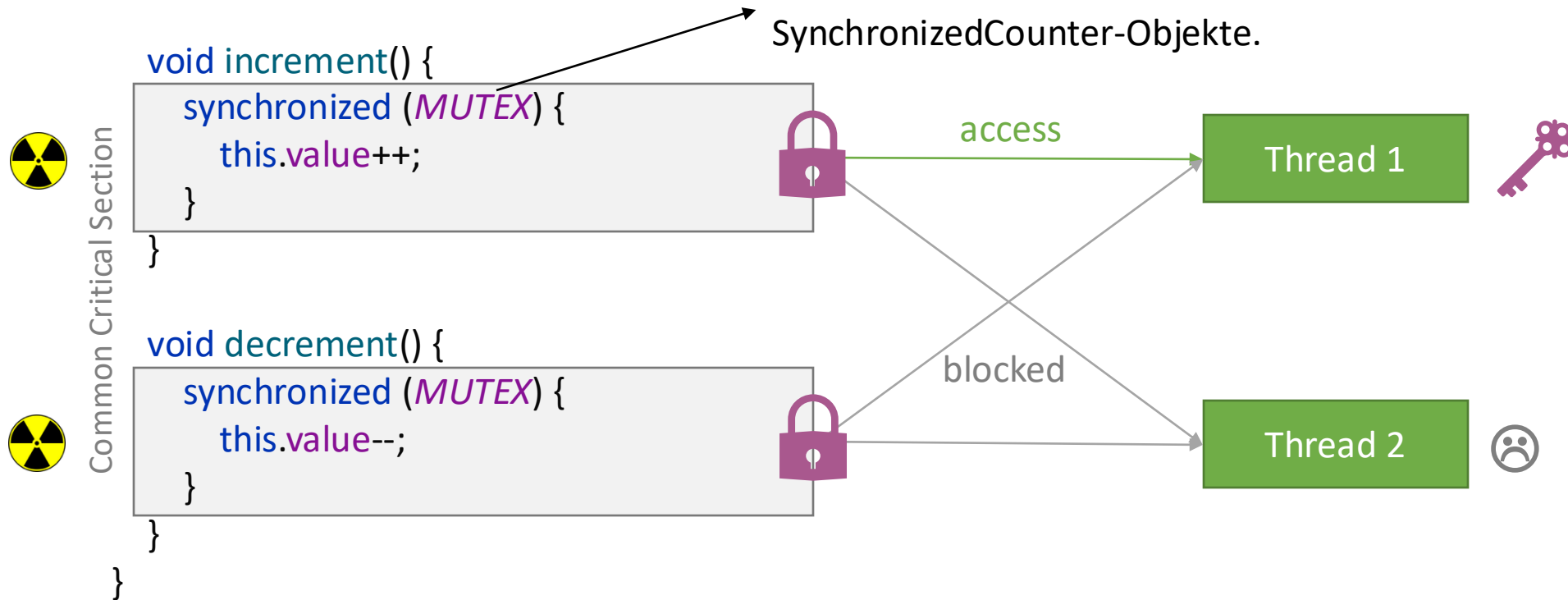
Gemeinsamer kritischer Bereich: static

```
class SynchronizedCounter {  
    private static final Object MUTEX = new Object();  
    private int value = 0;
```

```
void increment() {  
    synchronized (MUTEX) {  
        this.value++;  
    }  
}
```

```
void decrement() {  
    synchronized (MUTEX) {  
        this.value--;  
    }  
}
```

Ein Schloss und Schlüssel für alle SynchronizedCounter-Objekte.



In diesem Beispiel ist die Mutex **static**, d. h. alle SynchronizedCounter-Objekte **teilen sich dasselbe Object **MUTEX****. Daraus folgt, dass es nur **ein einziges Schloss und einen einzigen Schlüssel** gibt. Dieses Beispiel ist nicht sinnvoll, da **für alle Objekte** maximal ein Thread **increment** oder **decrement** aufrufen kann, obwohl sich **value** für jedes SynchronizedCounter-Objekt unabhängig ändern ließe.

Mehrere kritische Bereiche

```
class SynchronizedCounters {  
    private final Object mutex1 = new Object();  
    private final Object mutex2 = new Object();  
    private int value1 = 0;  
    private int value2 = 0;
```

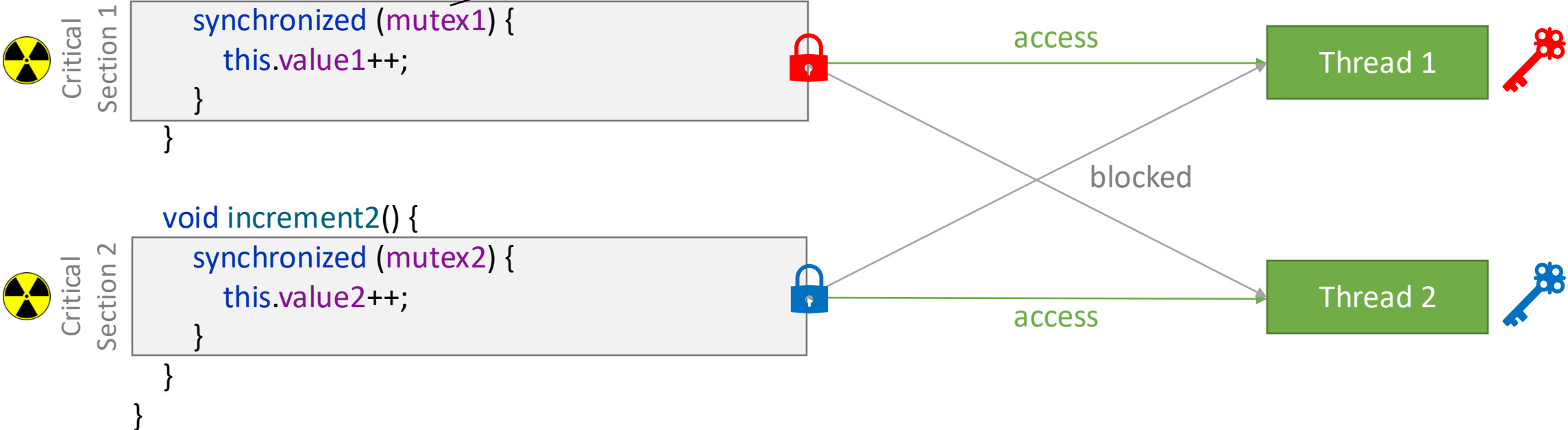
Critical Section 1

```
void increment1() {  
    synchronized (mutex1) {  
        this.value1++;  
    }  
}
```

Critical Section 2

```
void increment2() {  
    synchronized (mutex2) {  
        this.value2++;  
    }  
}
```

Zwei Schlösser und Schlüssel pro SynchronizedCounter-Objekt.



Mehrere Mutexe sind möglich, z. B., um die Parallelität zu erhöhen (bessere Performance).

Mehrere kritische Bereiche – Anomalie

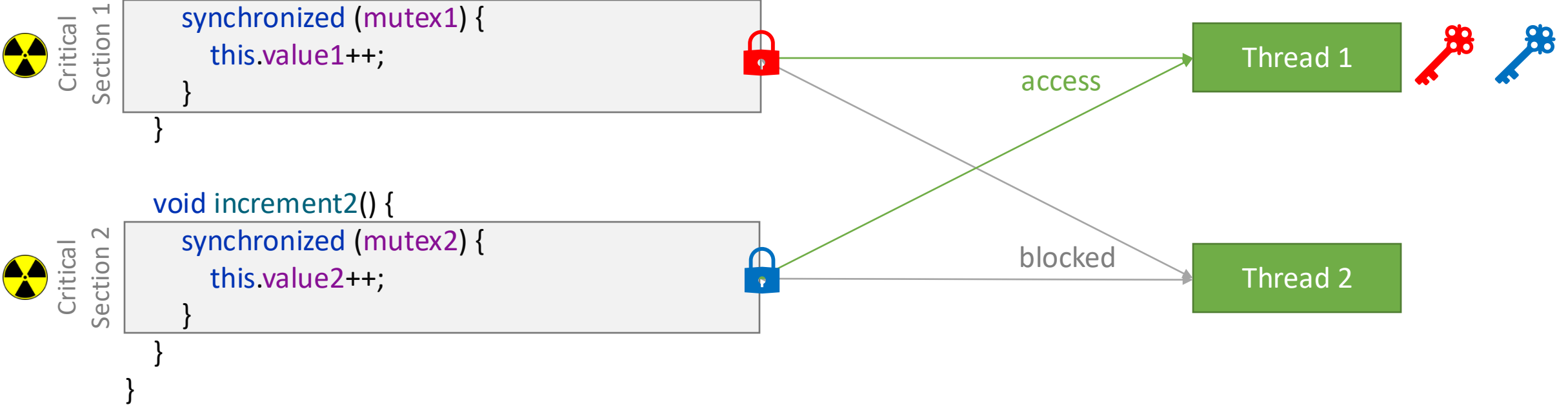
```
class SynchronizedCounters {  
    private final Object mutex1 = new Object();  
    private final Object mutex2 = new Object();  
    private int value1 = 0;  
    private int value2 = 0;
```

```
    void increment1() {
```

```
        synchronized (mutex1) {  
            this.value1++;  
        }  
    }
```

```
    void increment2() {
```

```
        synchronized (mutex2) {  
            this.value2++;  
        }  
    }
```



Was stimmt hier nicht? 🤔

Mehrere kritische Bereiche #WTF

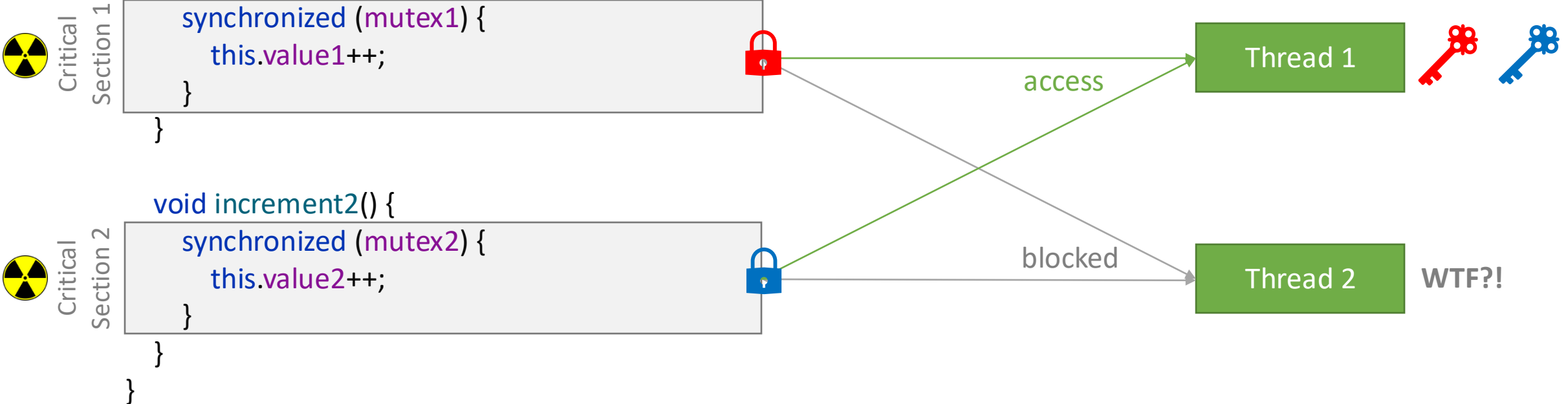
```
class SynchronizedCounters {  
    private final Object mutex1 = new Object();  
    private final Object mutex2 = new Object();  
    private int value1 = 0;  
    private int value2 = 0;
```

```
    void increment1() {
```

```
        synchronized (mutex1) {  
            this.value1++;  
        }  
    }
```

```
    void increment2() {
```

```
        synchronized (mutex2) {  
            this.value2++;  
        }  
    }
```



Dieses Beispiel ist **nicht möglich**, weil Thread 1 nicht beide Schlüssel gleichzeitig haben kann!
Thread 1 müsste zunächst den kritischen Bereich von `mutex1` beenden und freigeben, bevor Thread 1 den anderen kritischen Bereich von `mutex2` anfordern und betreten kann.

Lost Update

```
class Lost {  
  
    private static final int TO_TEN = 10;  
    private static final Counter sharedCounter = new Counter(0); // mutex  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_TEN));  
        var t2 = new Thread(count(TO_TEN));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                 synchronized (sharedCounter) { // synchronized schützt sharedCounter vor Race Conditions!  
                    sharedCounter.increment(); // value++;  
                }  
                System.out.println(sharedCounter.value());  
            }  
        };  
    }  
}
```

Ein **Lost Update** ist ein spezieller Fall von Race Condition, bei dem eine oder mehrere **Aktualisierungen** durch **sofortiges Überschreiben** der **gemeinsam genutzten Ressource** **verloren** gehen.

Mögliche Ausgabe:

1
2
4
4
5
6
8
8
9
10
12
12
14
14
16
16
17
18
20
20

Es kommt immer 20 am Ende raus,
aber es fehlt z. B. die 19. #WTF?! 47

Lost Update: In Detail

Thread 1

```
synchronized (SHARED_COUNTER) {  
    SHARED_COUNTER.increment();  
}  
// Threadwechsel zu t2  
System.out.println(SHARED_COUNTER.value());
```

Thread 2

```
synchronized (SHARED_COUNTER) {  
    SHARED_COUNTER.increment();  
}  
// Threadwechsel zu t1  
System.out.println(SHARED_COUNTER.value());
```

Thread 1 führt `SHARED_COUNTER.increment();` aus

Thread 1 wird direkt nach `synchronized (SHARED_COUNTER)` von Thread 2 unterbrochen.

Thread 2 ist dran und führt auch `SHARED_COUNTER.increment();` aus.

→ `SHARED_COUNTER` wurde **zweimal erhöht**, aber **ein neuer Wert wurde nicht gelesen!**

Beide Threads lesen anschließend `SHARED_COUNTER` und geben den **letzten** Wert aus.

→ Werte werden doppelt ausgegeben und andere Werte fehlen.

Immerhin: Am Ende kommt immer 20 heraus! 🤖

Mögliche Ausgabe:

1
2
4
4
5
6
8
8
9
10
12
12
14
14
16
16
17
18
20
20

Lost Update: Atomic Section

```
class LostAtomicSection {  
  
    private static final int TO_TEN = 10;  
    private static final Counter SHARED_COUNTER = new Counter(0); // mutex  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_TEN));  
        var t2 = new Thread(count(TO_TEN));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                synchronized (SHARED_COUNTER) { // synchronized schützt vor Race Conditions.  
                    SHARED_COUNTER.increment(); // value++;  
                    System.out.println(SHARED_COUNTER.value()); // println ist jetzt mit im atomaren Bereich.  
                                                                // Der atomare Bereich kann nicht durch  
                                                                // andere Threads unterbrochen werden.  
                }  
            }  
        };  
    }  
}
```



Ausgabe:

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20



Lost Update: AtomicInteger

```
class LostAtomicInteger {  
  
    private static final int TO_TEN = 10;  
    private static final AtomicInteger SHARED_COUNTER = new AtomicInteger(0); // AtomicInteger ist thread-safe  
  
    public static void main(String[] args) throws Exception {  
        var t1 = new Thread(count(TO_TEN));  
        var t2 = new Thread(count(TO_TEN));  
  
        t1.start();  
        t2.start();  
  
        t1.join(); // wait for t1 to finish.  
        t2.join(); // wait for t2 to finish.  
    }  
  
    private static Runnable count(int limit) { // DRY  
        return () -> {  
            for (int i = 1; i <= limit; i++) {  
                System.out.println(SHARED_COUNTER.incrementAndGet()); // atomare Operation  
            }  
        };  
    }  
}
```



Statt unsere Klasse Counter und `synchronized` könnte man auch [java.util.concurrent.AtomicInteger](https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/AtomicInteger.html) als `SHARED_COUNTER` verwenden.

Ausgabe:

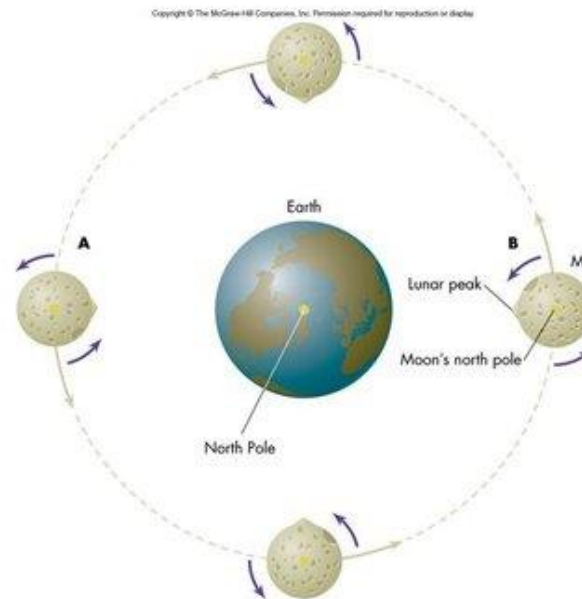
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20



Synchronized everything?



Warum nicht einfach **alles** **synchronized** machen?



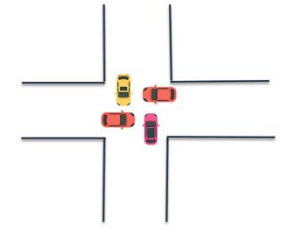
- The Moon keeps the same face toward the Earth as it orbits
- The fact that the Moon rotates at the same rate as it orbits the Earth is called ***synchronous rotation***

Synchronized: Minimalism

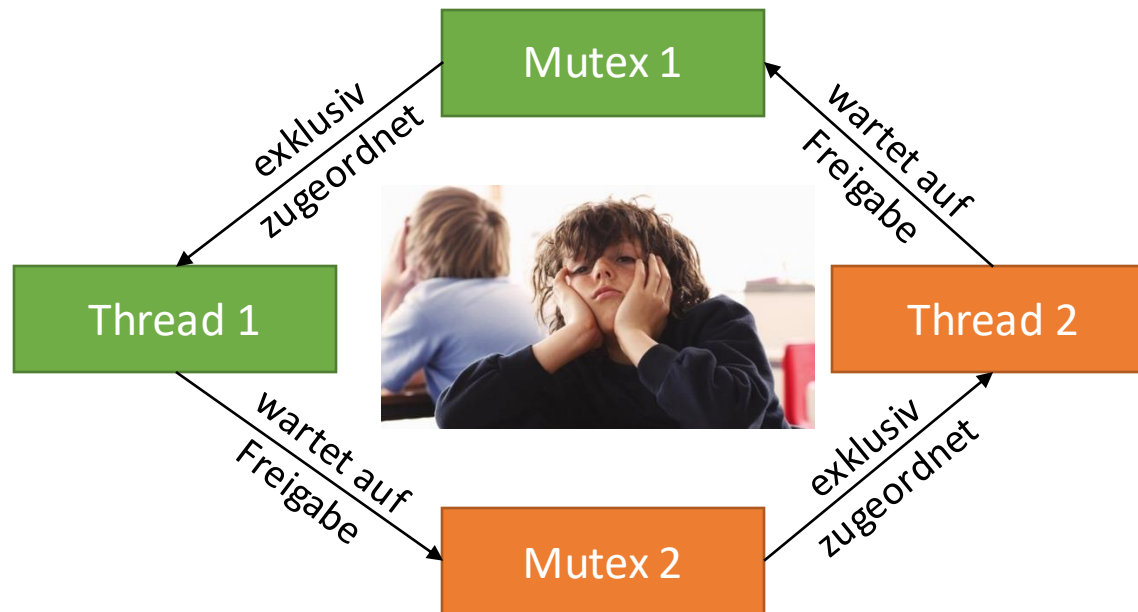
Synchronisierung ist nicht umsonst:

1. Es muss über das „**richtige**“ **Objekt synchronisiert** werden.
→ Fehlerhaftes Synchronisieren möglich.
2. Jeder synchronisierte Bereich hat einen gewissen **Overhead**.
→ Es muss immer geprüft werden, ob ein Thread rein darf oder nicht.
3. Ein synchronisierter Bereich zwingt andere Threads dazu, sich **anzustellen und zu warten**, bis sie an der Reihe sind. Je größer der kritische Bereich, desto länger die Wartezeit und desto weniger Parallelität.
→ **Synchronisierte Bereiche möglichst klein halten und nur dort, wo nötig.**
D. h. **wo gemeinsamer Schreibzugriff** stattfindet – **verwende möglichst immutable Objects!**
4. Synchronisierte Bereiche können zu **Deadlocks** führen!

Deadlock



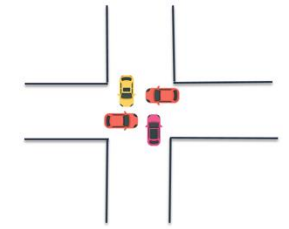
Ein **Deadlock** entsteht, wenn zwei (oder mehr) Threads **exklusiven Zugriff auf eine Ressource halten** (z. B. mittels `synchronized`), **die von jeweils anderem Thread benötigt wird**, um weitermachen zu können. Man nennt dies auch **zirkuläre Abhängigkeit**.



```
synchronized (mutex1) {  
    System.out.printf("tic ");  
    synchronized (mutex2) {  
        System.out.printf("toc ");  
    }  
}
```

Deadlocks treten in der Regel auf, wenn **synchronized-Blöcke** ineinander **verschachtelt** werden (möglichst vermeiden).

Deadlock: Beispiel



```
class Deadlock {
```

```
    public static void main(String[] args) throws InterruptedException {  
        final Object mutex1 = new Object();  
        final Object mutex2 = new Object();
```

```
  
        Thread t1 = new Thread(run(mutex1, mutex2));  
        Thread t2 = new Thread(run(mutex2, mutex1));  
        t1.start();  
        t2.start();  
        t1.join();  
        t2.join();
```

```
  
        System.out.println("Ende");  
    }
```

```
    private static Runnable run(Object mutexA, Object mutexB) {  
        return () -> {  
            synchronized (mutexA) {  
                System.out.println("mutex A");  
                sleep();  
                synchronized (mutexB) {  
                    System.out.println("mutex B");  
                    sleep();  
                }  
            }  
        }  
    };  
}
```

```
    private static void sleep() { try { Thread.sleep(100); } catch (InterruptedException e) { throw new RuntimeException(e); } }
```

Das Programm läuft endlos und wird nie beendet.

Hinweis: Thread::sleep dient hier dazu, eine Bearbeitungszeit zu simulieren, um die Wahrscheinlichkeit für einen Deadlock zu erhöhen. Ohne Thread::sleep ist es wahrscheinlich, dass einer der beiden Threads zu schnell ist und sowohl mutexA als auch mutexB reserviert. In dem Fall würde kein Deadlock auftreten.

Ausgabe:

```
mutex A  
mutex A
```

Code Smell: Thread::sleep

```
class SmellyThreads {  
    public static void main(String[] args) {  
        List<Integer> numbers = new LinkedList<>(List.of(3, 2, 1, 5, 4));
```

```
        Thread addRandomNumberThread = new Thread(() -> {  
            synchronized (numbers) {  
                numbers.add(randomElementFrom(numbers));  
            }  
        });
```

```
        Thread sortThread = new Thread(() -> {  
            sleep(Duration.ofMillis(50)); // Make sure the add-thread runs first.  
            synchronized (numbers) {  
                Collections.sort(numbers);  
            }  
        });
```

```
        Thread printThread = new Thread(() -> {  
            sleep(Duration.ofMillis(100)); // Make sure the other threads run first.  
            synchronized (numbers) {  
                for (var number : numbers) {  
                    System.out.print(number);  
                    sleep(Duration.ofMillis(100)); // For nice delayed printing effect.  
                }  
            }  
        });
```

```
        addRandomNumberThread.start();  
        sortThread.start();  
        printThread.start();  
    }  
}
```

zzZZzz

```
private static void sleep(Duration duration) {  
    try {  
        Thread.sleep(duration.toMillis());  
    } catch (InterruptedException e) {  
        throw new RuntimeException(e);  
    }  
}  
  
private static <T> T randomElementFrom(List<T> elements) {  
    return elements.get(randomIndexFor(elements));  
}  
  
private static final Random RANDOM = new Random();  
  
private static <T> int randomIndexFor(List<T> elements) {  
    return RANDOM.nextInt(0, elements.size());  
}  
}
```

Verwende **kein Thread::sleep**, um die **Ausführungsreihenfolge** von Threads **zu steuern**! Verwende stattdessen Thread:: join(), wait()/notify() oder FutureTask<V>.

Verwende **kein Thread::sleep innerhalb von synchronized**, da dadurch der Prozessorkern mit Schlafen verschwendet wird!

Zusammengefasst: Verwende Thread::sleep nur für Testzwecke.

Edsger Wybe Dijkstra



Dijkstra-Algorithmus: Den kennt fast jeder (kürzester Pfad im Graphen, nutzt jedes Navi).

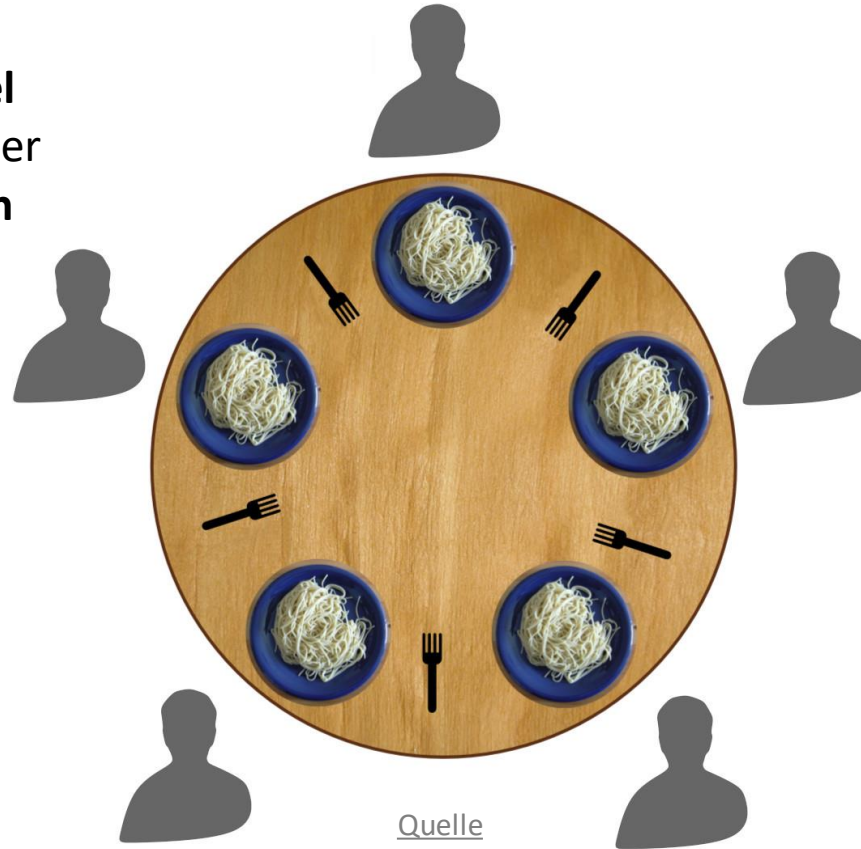
"Go To Statement Considered Harmful": Er hat maßgeblich dafür gesorgt, dass wir heute strukturiert programmieren (while, if, functions) und keinen wilden Spaghetti-Code mit GOTO-Befehlen mehr schreiben.

Nebenläufigkeit: Er hat als einer der Ersten das Problem der **Deadlocks** formalisiert und Konzepte wie **Semaphore** (und eben auch das Philosophen-Problem) erfunden, um Synchronisation mathematisch greifbar zu machen.

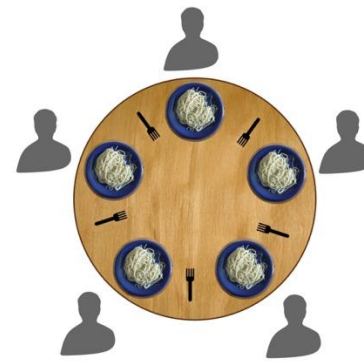
Deadlock: Dining Philosophers

Fünf Philosophen sitzen an einem runden Tisch. Zwischen jeweils zwei Philosophen liegt **eine Gabel** (insgesamt also fünf Gabeln). Jeder Philosoph kann entweder **denken** oder **essen**. Zum Essen benötigt ein Philosoph **zwei Gabeln** – die linke und die rechte.

Problem: Wenn alle Philosophen gleichzeitig ihre linke Gabel nehmen, kann keiner die rechte greifen – es entsteht ein **Deadlock** (Verklemmung).



Deadlock: Semaphore



Das Problem der Dining Philosophers lässt sich unterschiedlich lösen.

Idee: Asymmetrisches Gabel-Aufnehmen

Dies ließe sich **thread-safe** durch eine [Semaphore](#) umsetzen. Eine Semaphore ist eine Klasse, die einen **Zählerwert** speichert. Sie wird durch zwei **atomare, blockierende** Methoden gesteuert:

- **acquire()** – auch **wait()**, **P()**, **down()** genannt – **verringert** den Zähler
- **release()** – auch **signal()**, **V()**, **up()** genannt – **erhöht** den Zähler

// Pseudocode

```
Semaphore[5] forks = { new Semaphore(1), new Semaphore(1), new Semaphore(1), new Semaphore(1), new Semaphore(1) };
```

```
void philosopher(i) {                                     // Diese Methode wird für jeden Philosophen (0–4) als separater Thread gestartet.
    while(true) {
        denken();
        if i == 4 {                                       // Der Letzte Philosoph
            forks[(i+1) % 5].acquire(); // Nimm zuerst die rechte Gabel
            forks[i].acquire();           // Dann die linke Gabel
        } else {                                         // (alle anderen Philosophen)
            forks[i].acquire();           // Nimm zuerst die linke Gabel
            forks[(i+1) % 5].acquire(); // Dann die rechte Gabel
        }
        essen();
        forks[i].release(); // Gib linke Gabel frei
        forks[(i+1) % 5].release(); // Dann die rechte Gabel
    }
}
```

Hinweis: Diese Lösung kann zu Starvation (Verhungern) führen, d. h. , dass ein Philosoph nie beide Gabeln bekommt.